Guide to Python Basics

Python Introduction Notebook Summary

Fundamental concepts, tools, and syntaxes of Python programming language.

! Scope: This document is a summary of the "Python Intro" notebook from the course "AI and Deep Learning: Neural Network Applications" by MIT xPRO. This summary offers a quick overview of the main aspects included in the notebook.

Table of Contents

1. Variable and Values
2. Loops
3. Conditionals
4. Functions

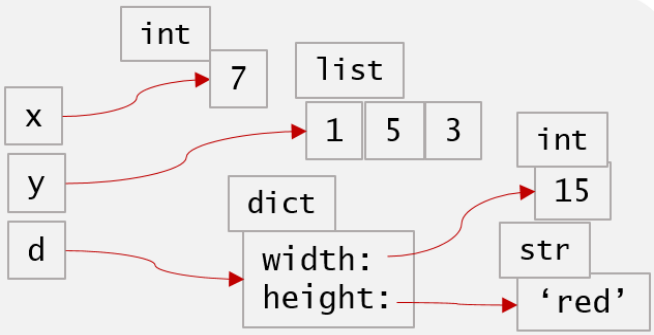
1. Variable and Values

Assigning value (the result of an evaluating expression) to a variable is a way to store values.

In Python, the argument of a variable follows the structure as seen below:

```
#<variable_name> = <expression>

#Integer:
x = 7
#List:
y = [1, 5, 3]
#Dictionary with integer & string:
d = {'width': 15,
     'color': 'red'}
```



! Note: In Python, lines of code that start with hash key # are comments made by the user and are disregarded by the programming language.

2. Loops

Loops are used to iterate through a data structure or repeat the same code multiple times.

- Execute code `while` a condition is `true` (Boolean).

```
#while <condition>:
      #<code body>

#Example using variables declared above in this document:
x = 7
while x < 8:
      x = x + 1
      print(x)
#Result from print:
8
```

! Note: This code prints `8` because the while loop starts with `x = 7` and runs `x = x + 1` while `x < 8`.

- Execute `for` statement to iterate over several values.

```
#for <var> in <iterable_values>:
    #<code body>

#Example using variables declared above in this document:
y = [1, 5, 3]
for value in y:
    print(value)
#Results from print:
1
5
3
```

! Note: This code prints `1,5,3` because the `for` loop iterates through the list variable `y = [1, 5, 3]` until the code does not find more values to print.

! Important: Indentation is relevant in Python, as it directs the scope of variables and scripts.

3. Conditionals

Conditionals statements are used for repeating alternative codes multiple times based on specific conditions:

"If X condition is met, execute the program in a predetermined way; If X condition is not met, do it another way."

```
#if <conditional>:
   #<code>
#elif <conditional>:
   #<code>
#else:
   #<code>

#Example:
if 5 > 10:
   print('first')
elif 2 + 3 > 4:
   print('second')
else:
   print('last')
#Result from print:
second
```

! Note: This code prints `second` because the second condition is met. That is, `5 > 4`

4. Functions

A function provides an efficient way to package and reuse and execute code in several different places in the user's program without having to copy and paste lines of code each time.

The user may create functions or call upon a previously-defined function:

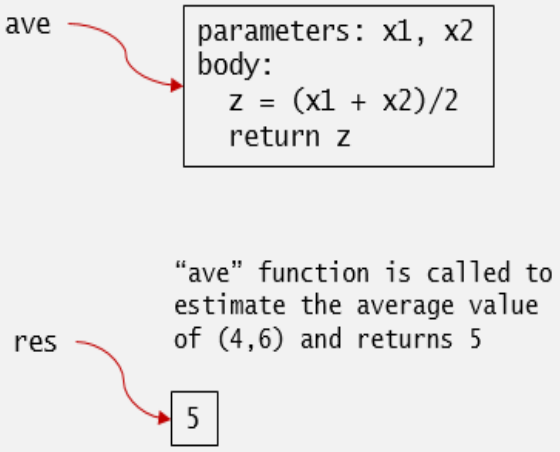
- `Define` a function with given a name, parameters, and body of the code
- Call upon the function for current arguments to run the body of the code and return a result

```
#Defining a function code structure:
#def <func_name>(<parameters>):
    #<code body>

#Example of defining a function:
def ave(x1, x2):
    z = (x1 + x2)/2.0
    return z

#Calling a function code structure:
<var> = <func_name>(<args>)


#Example of calling the function "ave"
defined above:
res = ave(4, 6)
print(res)
#Result from print:
5
```



! Note: Python's built-in libraries and external packages include many functions the user can call upon without writing the code itself.

In the code above the divisor `2.0` indicates that Python will return a value with decimals. If the user divides by `2` the return value will be an integer.